

Assignment

CSL 202: Design and Analysis of Algorithms

Akshat Gupta 12340160

Question 1:

For this question, you'll need the following definition:

- **Definition:** A sunlet is a graph with $2n$ vertices that consists of a cycle of length n , and each vertex in the cycle is directly connected to exactly one node of degree one. An example of a sunlet with 12 vertices is shown below.

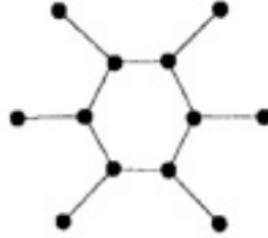


Figure 1: Example of sunlet

- **Model:** Input is an undirected graph G with $2n$ vertices. The algorithm can query an edge (i, j) and it will be told whether or not edge (i, j) is in graph G . Each query has cost 1.
- **Problem:** Output “Yes” if the input graph G is a sunlet, otherwise output “No”.
- **Your task:** Prove that every correct algorithm for the problem has worst-case cost at least $\binom{2n}{2}$.

Proof. Suppose, for the sake of contradiction, that there is an algorithm $\mathcal{A}$ whose worst-case cost is strictly less than $\binom{2n}{2}$. Then $\mathcal{A}$ examines only a strict subset of the $\binom{2n}{2}$ possible edges. Let

$$e = \{u, v\}$$

be any edge that $\mathcal{A}$ never queries (such an e exists since $\binom{2n}{2} - 1$ queries leave at least one unqueried pair).

Consider two candidate graphs on the same vertex set:

1. $G = S$, the *canonical* sunlet.
2. $G' = S \triangle \{e\}$, obtained by toggling the presence of edge e in S .

On every query $(i, j) \neq e$ that $\mathcal{A}$ makes, the answer in G and in G' is identical. Thus $\mathcal{A}$ receives the same transcript of answers on both inputs. But G is a sunlet, whereas G' is not (toggling e either breaks the cycle structure or creates a vertex of incorrect degree). Hence no algorithm that fails to query e can distinguish G from G' , contradicting the correctness of $\mathcal{A}$.

Therefore, every correct algorithm must query every edge, and the worst-case cost is at least

$$\binom{2n}{2}.$$

Problem 2.a:

Consider the following model and problem:

Model: For the range of numbers $1, 2, \dots, n$, there is a special threshold value $t \in \{0, 1, \dots, n\}$. For all numbers $i > t$, the number i is considered “unsafe”. All other numbers in the range $1, 2, \dots, n$ are considered “safe”. The algorithm can query any number $i \in \{1, 2, \dots, n\}$, and it will be told whether i is “safe” or “unsafe”. However, if the algorithm ever queries an “unsafe” i , the system shuts down and no further queries are possible.

Problem: Determine the exact value of t .

Your task:

- (a) Prove that any algorithm that solves the problem must perform at least n queries in the worst case. [5]

Proof. 1. Decision-tree perspective. Consider any algorithm as a decision tree: each internal node is labeled by a query $i \in \{1, \dots, n\}$, with two outgoing edges:

- “safe” branch (answer = yes), continuing the execution, and
- “unsafe” branch (answer = no), which immediately halts the process and outputs a guess for t .

2. Adversary strategy. The adversary picks $t = n$ (the largest possible). On each query $i \leq n$, it answers “safe.” Thus the algorithm sees no unsafe answers until it eventually queries all values up to n .

3. Lower-bounding the path length. Suppose an algorithm claimed to succeed with fewer than n queries in the worst case, say $k < n$. Then along every root-to-leaf path in its decision tree it makes at most k queries. Under the adversarial responses (always “safe”), the execution follows some path of length $\leq k$. After these $k < n$ safe answers, the algorithm has only seen that

$$t \geq \max\{\text{queried } i\} = m \leq k < n,$$

so it only knows $t \in \{m, m + 1, \dots, n\}$, which contains more than one possibility. Therefore it cannot pinpoint the exact value of t . This contradicts correctness.

4. Conclusion. Hence no algorithm can determine t with fewer than n queries in the worst case. In other words, the worst-case query complexity is $\Omega(n)$, and in fact exactly n .

□

Problem 2.b:

- (b) Let's change the model a bit: suppose that one "unsafe" query is allowed. That is, the system shuts down after exactly two "unsafe" queries. Prove that any algorithm that determines the exact value of t must use $\Omega(\sqrt{n})$ queries in the worst case. [7]

Proof. We use a *decision-tree* (adversary) argument. View any algorithm as a binary tree whose internal nodes are labeled by the index i queried, and whose edges correspond to the two possible answers:

- **Safe** ("yes"), and
- **Unsafe** ("no").

The model permits at most two unsafe answers: the first unsafe lets the algorithm continue; the second triggers shutdown (but its occurrence may still be used to deduce t).

Let k be the worst-case number of queries used by the algorithm. Each root-to-leaf path in the decision tree is a sequence of exactly k queries, with at most two of the answers being Unsafe. How many distinct root-to-leaf answer patterns are there? It is the number of binary strings of length k with at most two "no"s, namely

$$\sum_{j=0}^2 \binom{k}{j} = 1 + k + \binom{k}{2} = 1 + k + \frac{k(k-1)}{2} \leq 1 + k + \frac{k^2}{2}.$$

Hence the total number of leaves in the decision tree is at most

$$L(k) = 1 + k + \frac{k(k-1)}{2} \leq 1 + k + \frac{k^2}{2}.$$

On the other hand, the algorithm must correctly distinguish all $n+1$ possible values of t (from 0 through n), so the tree must have at least $n+1$ distinct leaves. Therefore

$$L(k) \geq n + 1.$$

Combining, we get:

$$1 + k + \frac{k^2}{2} \geq n + 1 \implies \frac{k^2}{2} \geq n \implies k \geq \sqrt{2n}.$$

Thus $k = \Omega(\sqrt{n})$, as claimed. □

Problem 2.c:

- (c) Design an algorithm that uses $O(\sqrt{n})$ many queries for the problem in part (b). [3]

Algorithm Idea (Block Search):

We partition the numbers 1 to n into blocks of size $\lceil \sqrt{n} \rceil$. The algorithm proceeds in two phases:

1. In the first phase, we find the block in which the threshold t lies by querying one number per block.
2. In the second phase, we perform a linear search within that block to determine t exactly.

Algorithm Description:

Let $m = \lceil \sqrt{n} \rceil$.

1. First Pass – Block Probing:

- For $i = m, 2m, 3m, \dots$, query index i .
- Stop when the first unsafe query is encountered (say at index im).
- Then t lies in the block $[(i-1)m + 1, im - 1]$.

2. Second Pass – Linear Search in Block:

- Linearly query all values in the range $[(i-1)m + 1, im - 1]$.
- Let j be the largest index such that j is safe.
- Then $t = j$.

3. Return t .

Correctness:

- The first unsafe query guarantees that $t < im$ and that all previous queries were safe.
- Therefore, t must be in the block before, which contains at most m elements.
- The linear search through that block will find the exact value of t .
- Only one unsafe query is ever used — the algorithm never causes a second unsafe query.

Query Complexity

- First phase: at most $\lceil \sqrt{n} \rceil$ queries.
- Second phase: at most $\lceil \sqrt{n} \rceil$ queries.
- Total: $O(\sqrt{n})$ queries.

Conclusion

The described algorithm uses only $O(\sqrt{n})$ queries in the worst case and ensures correctness while staying within the constraint of at most two unsafe queries.

Worst-case query complexity: $O(\sqrt{n})$

□